HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

MÔN HỌC: QUẢN LÝ DỰ ÁN PHẦN MỀM

ĐỀ TÀI

XÂY DỰNG DỰ ÁN PHẦN MỀM BẰNG ĐỘI AI AGENT ỨNG DỤNG CHO HỆ THỐNG QUẢN LÝ THỰC TẬP SINH (IMS)

Giảng viên hướng dẫn: ……

Sinh viên thực hiện: Trần Việt Anh

Mã sinh viên: …… Lớp: ……

Hà Nội, 2026

MỤC LỤC

1. Giới thiệu đề tài và yêu cầu

2. Tổng quan hệ thống IMS

3. Bộ công cụ AgentKit

   3.1. Khái niệm

   3.2. Triển khai AgentKit trong dự án IMS

4. Tổ chức đội AI Agent

5. Quy trình triển khai dự án bằng agent

6. Cơ chế điều phối và kiểm soát chất lượng

7. Triển khai thực tế (chạy thật)

   7.1. Chuẩn bị môi trường và cài AgentKit

   7.2. Vòng 0: đánh giá hiện trạng

   7.3. Vòng 1: sửa lỗi và bổ sung kiểm thử

   7.4. Vòng 2: tính năng Mentor, CI và đo độ phủ

   7.5. Kiểm thử hồi quy, review lại và phát hành

   7.6. Vòng 4–6: gia cố khóa đăng nhập, kiểm thử đột biến và sửa CORS

   7.7. Ứng dụng chạy thật và video demo

8. Kết quả và đánh giá

9. Video demo

10. Kết luận và hướng phát triển

Phụ lục A. Hướng dẫn chạy

Phụ lục B. Danh sách báo cáo do agent tạo ra

Phụ lục C. Phiếu giao việc thực tế

1. Giới thiệu đề tài và yêu cầu

1.1. Bối cảnh

Các mô hình ngôn ngữ lớn hiện nay không chỉ trả lời câu hỏi mà còn có thể hành động: đọc mã nguồn, chạy lệnh, sửa file, chạy kiểm thử và viết báo cáo. Một AI agent là một phiên làm việc của mô hình được giao một vai trò, một bộ công cụ và một phạm vi trách nhiệm rõ ràng. Khi ghép nhiều agent với vai trò khác nhau (quản lý dự án, lập trình, kiểm thử, review…) và đặt chúng dưới một quy trình có cổng kiểm soát, ta có một đội phát triển phần mềm trong đó con người đóng vai trò chủ sản phẩm và người phê duyệt.

Đề tài áp dụng cách làm đó cho dự án IMS (Intern Management System), một hệ thống quản lý thực tập sinh dạng client–server. Bộ công cụ được dùng là AgentKit (kit engineer) chạy trên Claude Code. Báo cáo mô tả cách tổ chức đội agent, quy trình triển khai, cơ chế kiểm soát chất lượng và nhật ký chạy thật trên mã nguồn dự án, kèm số liệu đo được ở từng vòng.

1.2. Yêu cầu của đề bài

1. Xây dựng một dự án từ các AI Agent: agent PM, agent coding, agent reviewer, agent tester, agent designer…
2. Viết báo cáo.
3. Chạy thật một dự án (hoặc một module phần mềm).
4. Quay video demo.

Bảng 1. Mức độ đáp ứng yêu cầu đề bài

#	Yêu cầu	Cách đáp ứng	Vị trí
1	Đội AI Agent	16 agent của AgentKit, ánh xạ vào các vai trò PM, BA, Designer, Coder, Reviewer, Tester, DevOps, tài liệu	Chương 3–6
2	Báo cáo	Tài liệu này, sinh tự động từ mã nguồn bằng python-docx	Toàn bộ
3	Chạy thật	Agent chạy thật trên mã nguồn IMS qua ba vòng: đánh giá, sửa lỗi và bổ sung kiểm thử, thêm tính năng và CI; ứng dụng chạy thật với MySQL	Chương 7, 8
4	Video demo	Kịch bản chi tiết từng phân đoạn	Chương 9

2. Tổng quan hệ thống IMS

2.1. Mục tiêu và phạm vi

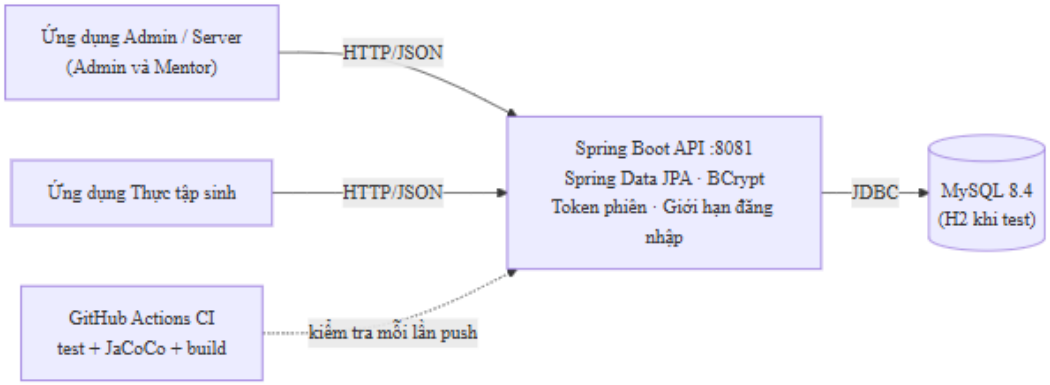
IMS số hóa quy trình quản lý thực tập sinh từ lúc tiếp nhận, phân công mentor, giao việc, báo cáo hằng tuần, đánh giá cho đến khi kết thúc kỳ thực tập. Phạm vi MVP gồm: đăng nhập và phân quyền ba vai trò; quản lý hồ sơ thực tập sinh; phân mentor và phòng ban; giao task và cập nhật trạng thái; nộp và chấm báo cáo tuần; đánh giá cuối kỳ; dashboard

thống kê. Ngoài phạm vi: chấm công GPS, tính lương, tuyển dụng, gửi email hay SMS thật, AI chấm điểm.

Bảng 2. Tác nhân của hệ thống

Vai trò	Chức năng chính
ADMIN (HR)	Quản trị hệ thống, cấp tài khoản, tạo/sửa/xóa hồ sơ thực tập sinh, quản lý phòng ban và mentor
MENTOR	Xem thực tập sinh được phân công, giao task, chấm báo cáo tuần, đánh giá cuối kỳ
INTERN	Xem task của mình, cập nhật TODO → IN_PROGRESS → COMPLETED, nộp báo cáo tuần, xem đánh giá

2.2. Kiến trúc và công nghệ



Hình 1. Kiến trúc client–server của IMS

Bảng 3. Công nghệ sử dụng

Tầng	Công nghệ	Thư mục
Backend	Spring Boot 3.3.4, Spring Data JPA, Bean Validation, BCrypt	app/backend
Frontend	React, Vite	app/frontend
Desktop	Electron: ứng dụng Admin/Server và ứng dụng Thực tập sinh	app/desktop, app/scripts
Cơ sở dữ liệu	MySQL 8.4 khi chạy thật; H2 (chế độ MySQL) khi kiểm thử	app/database
Tích hợp liên tục	GitHub Actions, JaCoCo	.github/workflows

2.3. Yêu cầu nghiệp vụ

Bảng 4. User story của IMS

Mã	User story	Tiêu chí nghiệm thu chính
US01	Đăng nhập theo vai trò	Sai thông tin trả 401; lưu token và vai trò
US02	Admin tạo hồ sơ thực tập sinh	Email không trùng; có phòng ban và mentor; ngày kết thúc sau ngày bắt đầu
US03	Mentor giao task	Chỉ giao cho thực tập sinh của mình; deadline trong kỳ thực tập; task mới ở TODO
US04	Thực tập sinh cập nhật task	Chỉ sửa task của mình; không nhảy thẳng TODO → COMPLETED
US05	Báo cáo tuần	Mỗi tuần một báo cáo; mentor chấm điểm và chuyển REVIEWED
US06	Đánh giá cuối kỳ	Điểm thành phần 0–10; điểm tổng là trung bình

3. Bộ công cụ AgentKit

3.1. Khái niệm

AgentKit là bộ công cụ biến Claude Code thành một đội phát triển nhiều agent. Bộ gồm hai phần. Phần thứ nhất là CLI ak, dùng để cài đặt, cập nhật, kiểm tra sức khỏe (ak doctor), quản lý kit, kế hoạch, nhật ký và bản sao lưu. Phần thứ hai là kit engineer, gói nội dung được cài vào dự án gồm agent, skill, rule và hook.

- **Agent:** nhân sự của đội. Mỗi agent là một file mô tả vai trò, mô hình AI được dùng và bộ công cụ được phép.
- **Skill:** quy trình chuẩn gọi bằng lệnh /ak-<tên>, ví dụ /ak-plan, /ak-cook, /ak-code-review, /ak-test.
- **Rule:** luật chung áp dụng cho mọi agent: quy trình, cách review, cách điều phối, quản lý tiến trình.
- **Hook:** script chạy tự động trước hoặc sau mỗi thao tác, là rào chắn mà agent không thể bỏ qua.

Bảng 5. Thành phần AgentKit đã cài vào dự án

Thành phần	Số lượng	Vị trí
Agent	16	.claude/agents/*.md
Skill	105	.claude/skills/
Rule	8	.claude/rules/*.md
Hook	18 script, 27 điểm gắn sự kiện	.claude/hooks/, .claude/settings.json
Cấu hình dự án	1	.agentkit/config.yaml

3.2. Triển khai AgentKit trong dự án IMS

Phần này mô tả từng bước đưa AgentKit vào kho mã QLDAPM như đã làm thật ngày 05/11/2026, gồm cả các sự cố gặp phải và cách xử lý. Môi trường: Windows 11, Claude Code, ak CLI 3.0.0.

Bảng 6. Các bước triển khai AgentKit trong dự án

Bước	Việc làm	Lệnh / thao tác	Kết quả
1	Kiểm tra hiện trạng	ak doctor; ak projects list	Chưa có kit nào; dự án chưa có trong registry (chỉ có dự án CSDL-DPT)
2	Khởi tạo dự án	ak init	Tạo .agentkit/ (lock, ownership.json, script-audit.json); đăng ký QLDAPM vào registry
3	Cài kit vào dự án	ak kit install engineer --yes	Registry từ xa trả 401 khi liệt kê kit; ak dùng bản kit engineer có sẵn trong cache cục bộ. Ghi 1869 file: .claude/ (agent, skill, rule, hook, settings.json) và .agentkit/config.yaml; 18 hook script
4	Kiểm tra sức khỏe	ak doctor	Cảnh báo 28 lỗi đăng ký hook, nằm ở bản cài chung (~/.claude) chứ không phải bản của dự án
5	Sửa bản cài chung	ak kit init engineer --remote --target claude-code --global --force	Sinh viên tự chạy (lệnh ghi đè cấu hình chung nên cần chính chủ cho phép). ak tự sao lưu (ak backups restore 20261105T025535Z-c8434883); ak doctor còn 0 lỗi
6	Nạp vào phiên làm việc	Mở lại phiên Claude Code	Hook SessionStart và UserPromptSubmit chạy; 8 rule được nạp; danh sách skill hiện các skill /ak-*
7	Quyết định quản lý mã nguồn	Thêm .claude/ và .agentkit/ vào .gitignore	Công cụ là cục bộ, không đưa lên kho; mọi đầu ra của agent (mã, test, báo cáo) đều commit

3.2.1. Cấu trúc thư mục sau khi cài

```
QLDAPM/
├── .agentkit/
│   ├── config.yaml        # cấu hình dự án: đường dẫn, tên plan, hook
│   └── adapters/…         # bản kit engineer đã cài
├── .claude/
│   ├── agents/            # 16 agent: tester, code-reviewer…
│   ├── skills/            # 105 skill: ak-plan, ak-cook, ak-test…
│   ├── rules/             # 8 rule áp dụng cho mọi agent
│   ├── hooks/             # 18 hook script (.cjs)
│   ├── agent-memory/      # bộ nhớ dự án của agent
│   └── settings.json      # đăng ký hook vào sự kiện Claude Code
├── app/                   # mã nguồn IMS
├── plans/reports/         # báo cáo do agent viết
└── docs/                  # báo cáo này, hình, video
```

3.2.2. Một agent được định nghĩa như thế nào

Mỗi agent là một file Markdown có phần đầu (frontmatter) khai báo tên, mô tả khi nào dùng, mô hình AI, bộ nhớ và danh sách công cụ được phép; phần thân là chỉ dẫn vai trò. Ví dụ trích từ .claude/agents/tester.md:

```
---
name: tester
description: 'Use this agent when you need to validate code quality through testing…'
model: haiku
memory: project
tools: Glob, Grep, Read, Edit, Write, Bash, PowerShell, …, Task(Explore), Task(kongming)
---
You are a **QA Lead** performing systematic verification of code changes. …
**Hard-problem escalation:** when stuck … consult the `kongming` agent
```

Nhờ khai báo tools, mỗi agent chỉ có đúng quyền cần thiết: ví dụ project-manager không có quyền chạy lệnh shell, git-manager không có quyền sửa file. Dòng Task(kongming) cho phép tester tự gọi cố vấn khi bế tắc.

3.2.3. Hook được nối vào Claude Code như thế nào

File .claude/settings.json đăng ký từng hook vào một sự kiện và một bộ lọc công cụ. Ví dụ hook đặt tên file chạy trước mỗi lần ghi file:

```
"PreToolUse": [{
  "matcher": "Write",
  "hooks": [{ "type": "command",
              "command": "node \"${CLAUDE_PROJECT_DIR}/.claude/hooks/descriptive-
name.cjs\"" }]
}]
```

Hook nhận thông tin thao tác sắp thực hiện; nếu vi phạm luật thì trả về lỗi chặn kèm lý do, nếu không thì có thể bổ sung ngữ cảnh. Hook UserPromptSubmit (dev-rules-reminder) chèn vào mỗi lượt yêu cầu: ngày giờ, múi giờ, đường dẫn plans/, docs/, quy ước đặt tên báo cáo {agent}-{YYMMDD-HHmm}-{slug}.md. Đó là lý do mọi báo cáo agent trong plans/reports/ có tên thống nhất như tester-261105-1002-ims-test-report.md.

3.2.4. Cách điều phối viên gọi agent và skill

- **Gọi agent:** điều phối viên dùng công cụ Agent của Claude Code với tham số subagent_type (ví dụ tester, code-reviewer), nội dung là phiếu giao việc, và chạy nền để nhiều agent làm song song. Agent con nhận ngữ cảnh dự án qua hook SubagentStart, làm xong gửi báo cáo về và kết thúc bằng dòng Status.
- **Gọi skill:** người dùng gõ /ak-<tên> (ví dụ /ak-code-review, /ak-test) hoặc điều phối viên tự chọn skill theo rule skill-domain-routing; nội dung SKILL.md được nạp làm quy trình cho lượt đó.

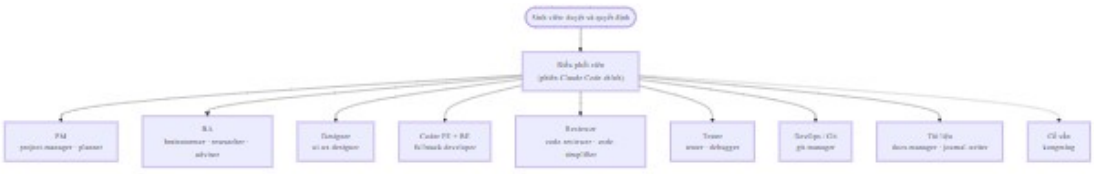
- **Vận hành:** ak doctor để kiểm tra sức khỏe, ak backups restore để quay lại, ak update để cập nhật kit, /ak-plans-kanban để xem bảng kế hoạch.

3.2.5. Cấu hình dự án

.agentkit/config.yaml là nơi đặt cấu hình dùng chung cho cả nhóm (được thiết kế để commit, không chứa khóa bí mật): mức độ giải thích (coding_level), đường dẫn docs và plans, giới hạn số dòng tài liệu, quy tắc đặt tên thư mục kế hoạch, ngôn ngữ phản hồi, bật tắt từng hook. Dự án IMS dùng giá trị mặc định: tài liệu ở docs/, kế hoạch và báo cáo ở plans/, tài liệu tối đa 800 dòng.

4. Tổ chức đội AI Agent

Kế hoạch gốc của dự án (app/docs/03-pm-plan.md) cần một đội năm người: PM, BA, FE, BE và Tester. Với AgentKit, mỗi vai trò do một hoặc vài agent đảm nhận. Một phiên Claude Code chính đóng vai trò điều phối viên: nhận yêu cầu, chia việc, giao phiếu cho agent, gom kết quả và báo lại. Sinh viên giữ vai trò chủ sản phẩm (Product Owner) và là người duy nhất ra các quyết định nghiệp vụ hoặc không thể đảo ngược.



Hình 2. Cơ cấu đội AI Agent

Bảng 7. Ánh xạ vai trò dự án sang agent AgentKit

Vai trò	Agent	Mô hình	Đầu ra
PM: quản lý tiến độ	project-manager	Opus	Theo dõi trạng thái kế hoạch, báo cáo tiến độ, điều phối giữa các phiên
PM: lập kế hoạch	planner	Opus	Kế hoạch nhiều phase trong plans/, có tiêu chí nghiệm thu
BA: phân tích yêu cầu	brainstormer, researcher, advisor	Opus, Fable	So sánh phương án, nghiên cứu công nghệ, phỏng vấn chốt phạm vi
Designer	ui-ux-designer	Opus	Wireframe, design system, bố cục responsive
Coder (FE + BE)	fullstack-developer	Opus	Mã nguồn theo phase với ranh giới file rõ ràng
Reviewer	code-reviewer, code-simplifier	Opus	Báo cáo review bảo mật, lỗi, khả năng bảo trì
Tester / QA	tester, debugger	Haiku, Opus	Chạy test, build, smoke API; truy nguyên nhân lỗi

DevOps / Git	git-manager	Haiku	Commit chuẩn conventional commits, push, quét bí mật
Tài liệu	docs-manager, journal-writer	Opus	Tài liệu dự án, nhật ký kỹ thuật
Cố vấn	kongming	Fable	Lời khuyên một lần cho quyết định khó, không viết mã
Trinh sát mã	explore	Haiku	Tìm file, lần quan hệ giữa các thành phần

4.1. Nguyên tắc chọn mô hình

- **Haiku** cho việc lặp lại, chi phí thấp: chạy kiểm thử, commit, tìm file.
- **Opus** cho việc cần suy luận: lập kế hoạch, viết mã, review.
- **Fable** chỉ dùng cho quyết định khó hoặc khi bế tắc, thông qua kongming hoặc advisor.

4.2. Skill tương ứng từng giai đoạn

Bảng 8. Skill AgentKit theo giai đoạn

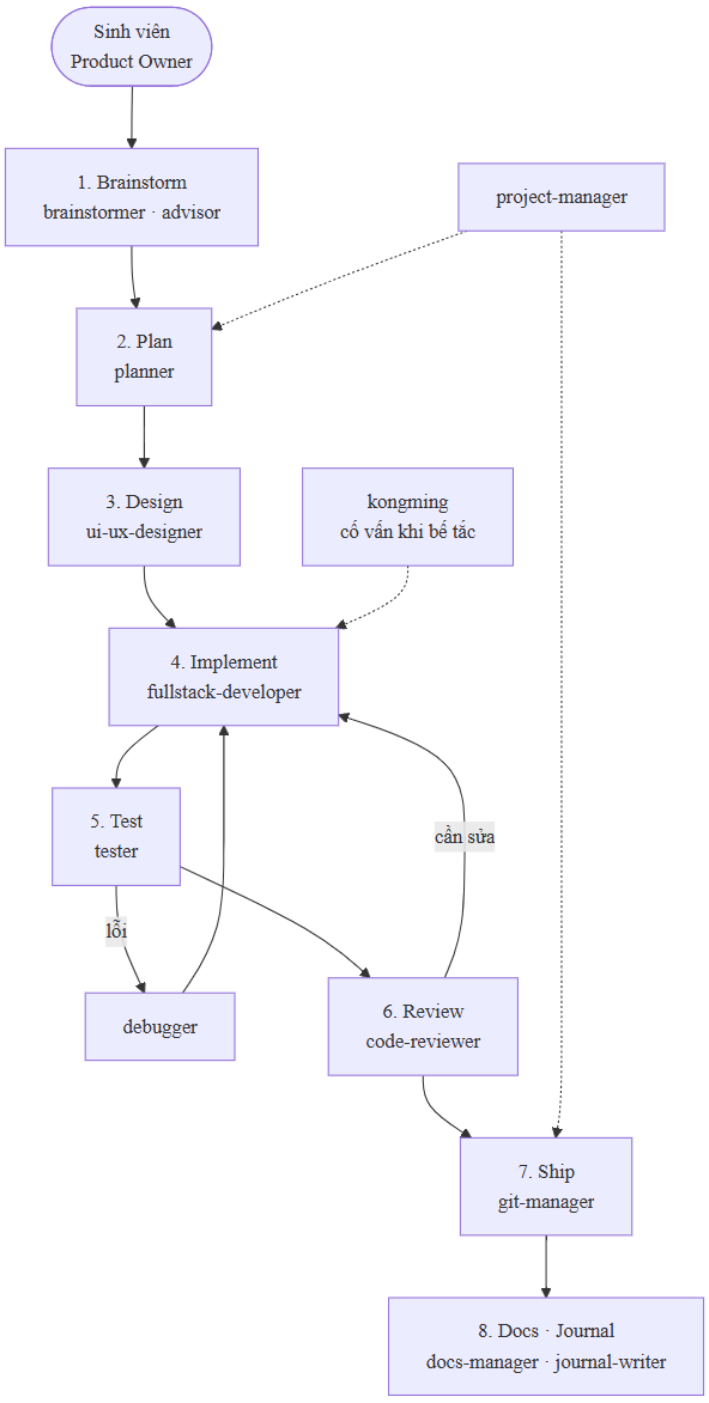
Giai đoạn	Skill	Việc làm
Chốt yêu cầu	/ak-brainstorm, /ak-advise	Biến ý tưởng thành kết quả, ràng buộc, tiêu chí nghiệm thu
Lập kế hoạch	/ak-plan	Kế hoạch nhiều phase, kiểm tra và phản biện kế hoạch
Thiết kế	/ak-frontend-design, /ak-ui-ux-pro-max, /ak-databases	Giao diện, design system, lược đồ dữ liệu
Lập trình	/ak-cook, /ak-backend-development, /ak-frontend-development	Hiện thực kế hoạch đã duyệt
Kiểm thử	/ak-test, /ak-web-testing, /ak-scenario	Chạy test, sinh ca kiểm thử biên
Sửa lỗi	/ak-debug, /ak-fix	Chứng minh nguyên nhân trước khi sửa
Review	/ak-code-review, /ak-security	Review theo bằng chứng, rà soát bảo mật
Phát hành	/ak-git, /ak-ship	Commit, push, mở PR, chờ CI
Tài liệu	/ak-docs, /ak-journal, /ak-sumup	Tài liệu, nhật ký, tổng kết
Quản lý	/ak-project-management, /ak-plans-kanban, /ak-watzup	Theo dõi tiến độ, bảng kanban
Đội nhiều phiên	/ak-team, /ak-orchestrate, /ak-vibe	Chạy song song nhiều agent, pipeline đến PR

5. Quy trình triển khai dự án bằng agent

5.1. Pipeline chuẩn

Quy trình được định nghĩa trong rule skill-workflow-routing và primary-workflow: brainstorm → plan → design → implement → test → review → ship → docs/journal. Kiểm thử thất bại thì chuyển cho debugger rồi quay về implement; review chưa đạt thì

quay lại implement. project-manager theo dõi xuyên suốt; kongming được gọi khi gặp quyết định khó.



Hình 3. Pipeline triển khai của đội agent

Quy trình sửa lỗi đi theo nhánh riêng: xác định kết quả mong muốn → trinh sát → chẩn đoán → chọn cách sửa → sửa → kiểm thử → review. Luật bắt buộc là phải chứng minh được nguyên nhân trước khi thay đổi hành vi.

5.2. Ánh xạ Sprint sang agent

Bảng 9. Kế hoạch Sprint thực hiện bằng đội agent

Sprint	Nội dung	Agent và skill chính	Cổng chất lượng
0	Chốt phạm vi, WBS, công nghệ	advisor (/ak-advise), planner (/ak-plan)	Kế hoạch có tiêu chí nghiệm thu, sinh viên duyệt
1	Đăng nhập, phân quyền, User/Department	fullstack-developer (/ak-cook), tester	mvn test pass, không còn lỗi Critical
2	Hồ sơ thực tập sinh, phân mentor	ui-ux-designer, fullstack-developer, code-reviewer	TC03–TC05 pass
3	Giao việc, luồng trạng thái, deadline	fullstack-developer, tester (/ak-scenario)	TC06–TC11 pass
4	Báo cáo tuần, đánh giá cuối kỳ	fullstack-developer, code-reviewer (/ak-security)	TC12–TC18 pass
5	Dashboard, hồi quy, phát hành	tester, git-manager (/ak-ship), docs-manager	Definition of Done đạt

5.3. Phiếu giao việc và giao thức trạng thái

Theo rule orchestration-protocol, mỗi lần giao việc điều phối viên gửi cho agent một phiếu gồm: nhiệm vụ; file cần đọc; file được phép sửa; tiêu chí nghiệm thu; ràng buộc; đường dẫn ghi báo cáo. Mục file được phép sửa là chìa khóa để nhiều agent chạy song song mà không ghi đè lên nhau. Agent kết thúc bằng một trong bốn trạng thái DONE, DONE_WITH_CONCERNS, BLOCKED, NEEDS_CONTEXT. Nếu agent báo BLOCKED hoặc NEEDS_CONTEXT, điều phối viên phải đổi ngữ cảnh hoặc cách làm, không giao lại y nguyên. Phụ lục C trích một phiếu giao việc thực tế.

6. Cơ chế điều phối và kiểm soát chất lượng

6.1. Rule

Bảng 10. Các rule của AgentKit

Rule	Nội dung chính
primary-workflow	Chốt kết quả → khảo sát → lập kế hoạch và làm → kiểm chứng → review và kết thúc
development-rules	KISS, DRY; không dùng dữ liệu giả để qua kiểm thử; không giấu lỗi; không commit bí mật
review-audit-self-decision	Không đảo quyết định đã kiểm chứng khi không có bằng chứng mới; không tự ý đảo quyết định của người dùng
orchestration-protocol	Phiếu giao việc, cô lập ngữ cảnh, chạy song song, giao thức trạng thái
process-management	Theo dõi và dọn tiến trình nền, không để tiến trình "ma" chiếm cổng
documentation-management	Chỉ cập nhật tài liệu khi hành vi, thiết lập hoặc kiến trúc thay đổi
skill-workflow-routing, skill-	Chọn skill theo giai đoạn và theo lĩnh vực

domain-routing	

6.2. Hook

Bảng 11. Hook tiêu biểu

Thời điểm	Hook tiêu biểu	Tác dụng
Trước khi chạy lệnh, đọc file	privacy-block, scout-block, context-firewall	Chặn đọc file nhạy cảm (.env), chặn quét thư mục build nặng
Trước lệnh mạng	secrets-inject	Đưa bí mật vào lệnh mà không in ra màn hình
Trước khi ghi file	descriptive-name	Buộc tên file mô tả rõ
Khi người dùng gửi yêu cầu	dev-rules-reminder, secret-output-guardrail	Nhắc luật, đường dẫn báo cáo, cấm in bí mật
Khi agent con bắt đầu	subagent-init, team-context-inject	Bơm ngữ cảnh dự án cho agent con
Khi dừng hoặc nén ngữ cảnh	session-state, precompact-capture	Lưu trạng thái để làm tiếp

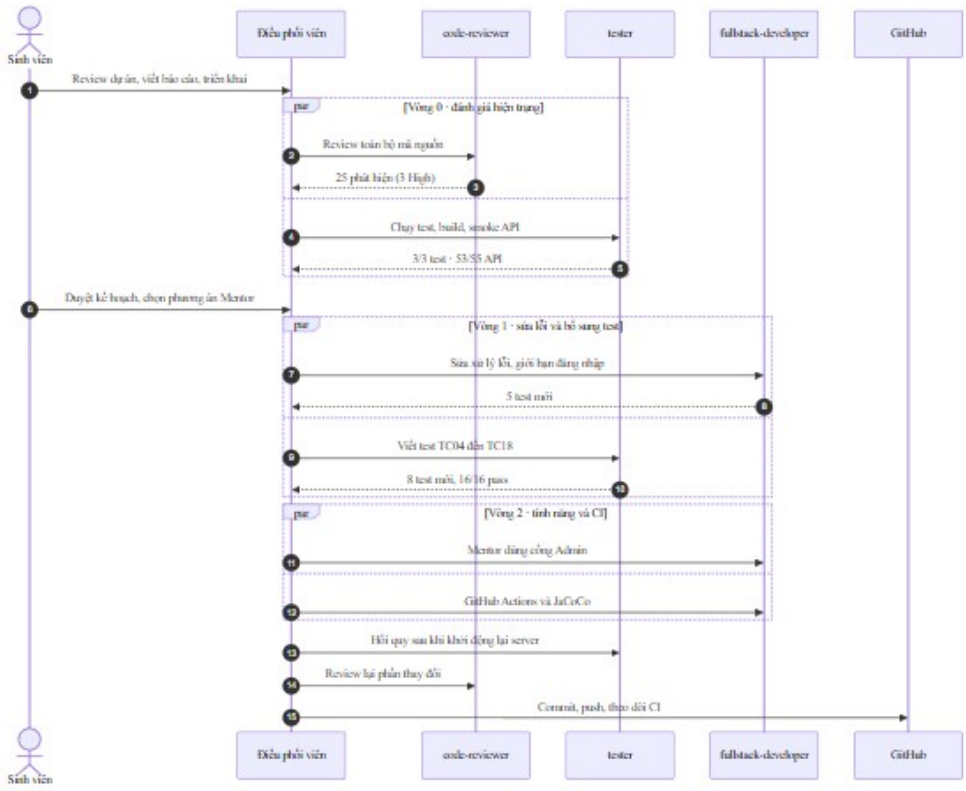
Ví dụ thật trong quá trình thực hiện: hook scout-block chặn lệnh chạy file jar trong thư mục build, điều phối viên chuyển sang lệnh chuẩn mvn spring-boot:run thay vì lách hook; hook privacy-block chặn agent code-reviewer đọc các file .env, và agent ghi rõ giới hạn đó trong báo cáo của mình.

6.3. Definition of Done khi làm việc với agent

5. tester báo build và kiểm thử pass kèm số liệu thô; không ghi "pass" khi chưa chạy.
6. code-reviewer không còn lỗi Critical hoặc High chưa xử lý trong phạm vi thay đổi.
7. Sinh viên đã xem và duyệt các quyết định nghiệp vụ.
8. Commit theo conventional commits, không chứa bí mật; CI trên GitHub xanh.
9. Tài liệu liên quan được cập nhật khi hành vi người dùng thay đổi.

7. Triển khai thực tế (chạy thật)

Chương này ghi lại những gì đã thực sự chạy ngày 05/11/2026. Mỗi vòng gồm một phiếu giao việc gửi cho agent, agent tự thực hiện (đọc mã, sửa mã, chạy lệnh) và trả về báo cáo có số liệu trong plans/reports/. Các agent trong cùng một vòng chạy song song, mỗi agent chỉ được ghi vào các file được giao để tránh xung đột.



Hình 4. Trình tự các vòng làm việc giữa sinh viên, điều phối viên và các agent

Bảng 12. Tóm tắt các vòng triển khai

Vòng	Mục tiêu	Agent (‖ = song song)	File thay đổi	Kết quả đo được
0	Đánh giá hiện trạng	code-reviewer ‖ tester	Không sửa mã	25 phát hiện (0/3/9/13); 3/3 test; smoke 53/55
1	Sửa lỗi xử lý ngoại lệ, giới hạn đăng nhập; bổ sung kiểm thử nghiệp vụ	fullstack-developer ‖ tester	ApiExceptionHandler, AuthService, App.jsx; 2 file test mới	16/16 test
2	Mentor dùng cổng Admin; CI và JaCoCo	fullstack-developer ‖ fullstack-developer	AuthService, AuthController, portal.js, App.jsx; pom.xml, ci.yml; 1 file test mới	19/19 test; độ phủ dòng 79,33%
3	Hồi quy, review lại, cập nhật tài liệu	tester ‖ code-reviewer ‖ docs-manager	5 file tài liệu của app	smoke 73/73; xem mục 7.5
4	Gia cố khóa đăng nhập (N1–N4); kiểm thử đột biến	fullstack-developer → code-reviewer ‖ tester	AuthService; 2 file test	25/25 test; đột biến 6/6
5	Khóa theo email tài khoản (N6); ghi rõ rủi ro N5	fullstack-developer	AuthService; 1 file test	26/26 test; review 0/1/9/13
6	Sửa CORS cổng 5174	fullstack-developer →	application.properties, .env.	26/26 test; độ phủ

	trong phiên được quay video	tester → code-reviewer	example	dòng 80,32%

7.1. Chuẩn bị môi trường và cài AgentKit

Bảng 13. Chuẩn bị môi trường

#	Việc	Kết quả
1	Đưa mã nguồn IMS vào kho mã, loại bỏ node_modules, thư mục build, file .exe khoảng 1,5 GB và CSDL mẫu	87 file trong app/; commit 4ec5e99
2	Build và kiểm thử backend	Máy chỉ có JDK 17 (pom khai báo 21); build bằng -Djava.version=17; 3/3 test pass
3	Chạy ứng dụng	Giao diện chặn khi backend dùng H2; chuyển sang MySQL 8.4 có sẵn, sinh viên duyệt tạo lại CSDL
4	Cài AgentKit	ak init, ak kit install engineer; ak doctor 0 lỗi

7.2. Vòng 0: đánh giá hiện trạng

Hai agent được giao song song. tester (mô hình Haiku) chạy kiểm thử backend, build frontend và một bộ smoke test chỉ đọc trên server thật; agent bị cấm ghi vào CSDL ngoài thao tác đăng nhập. code-reviewer (mô hình Opus) đọc toàn bộ mã nguồn, đối chiếu với tài liệu yêu cầu và hợp đồng API.

Bảng 14. Kết quả vòng 0

Hạng mục	Kết quả
Kiểm thử backend	3/3 pass (H2), 11,6 giây
Build frontend	Thành công, 36 module, JS 214,28 kB
Smoke API chỉ đọc	53/55 pass; 2 lỗi: đường dẫn không tồn tại trả 500 thay vì 404, sai method trả 500 thay vì 405
Review mã nguồn	25 phát hiện: 0 Critical, 3 High, 9 Medium, 13 Low; không có lỗi IDOR

Bảng 15. Ba phát hiện mức High

Mã	Phát hiện	Hậu quả
H1	Tài khoản mẫu mật khẩu 123456 và màn hình đăng nhập Admin in sẵn mật khẩu	Máy bất kỳ trong LAN lấy được quyền ADMIN chỉ bằng một request
H2	Chỉ có cổng ADMIN và INTERN; Mentor đăng nhập ở đâu cũng bị 403	US03, US05, US06 không dùng được qua giao diện
H3	Bộ xử lý ngoại lệ bắt mọi lỗi và không ghi log	404/405/415 thành 500; không debug được. Khớp với 2 lỗi tester tìm thấy

Đáng chú ý là lỗi H3 được hai agent tìm ra độc lập bằng hai cách khác nhau: tester qua việc gọi API thật, code-reviewer qua việc đọc mã. Điều phối viên còn tự đọc lại mã để xác minh cả ba lỗi High trước khi đưa vào kế hoạch. Quyết định cách xử lý H2 (cho Mentor dùng chung cổng Admin) do sinh viên đưa ra.

7.3. Vòng 1: sửa lỗi và bổ sung kiểm thử

- **fullstack-developer:** xử lý riêng NoResourceFoundException (404), HttpRequestMethodNotSupportedException (405, kèm header Allow), HttpMediaTypeNotSupportedException (415); ghi log lỗi bất ngờ. Thêm giới hạn đăng nhập: 5 lần sai trong 15 phút cho cùng một email thì trả 429. Bỏ hiển thị email và mật khẩu mẫu trên màn hình đăng nhập. Viết ErrorHandlingAndLoginLimitTest (5 test).
- **tester:** viết BusinessRuleTest (8 test) cho 7 test case trước đây chưa có kiểm thử tự động: TC04 email trùng, TC05 ngày kết thúc không sau ngày bắt đầu, TC08 deadline ngoài kỳ thực tập, TC10 nhảy thẳng TODO → COMPLETED, TC13 nộp trùng tuần, TC15 điểm ngoài 0–10, TC18 xem đánh giá của người khác. Luật đặt ra: nếu hành vi thật trái với tài liệu thì đánh dấu lỗi, không được nới lỏng assert. Kết quả: không có lỗi nghiệp vụ nào.

Kết quả: 16/16 test pass. Hai agent chạy song song mà không xung đột vì phiếu giao việc chia rõ file: một bên sửa mã nguồn, một bên chỉ tạo file test mới. Developer còn tự phát hiện rằng các lớp test dùng chung Spring context nên trạng thái khóa đăng nhập dùng chung, và thiết kế test để không khóa nhầm tài khoản mẫu.

7.4. Vòng 2: tính năng Mentor, CI và đo độ phủ

- **fullstack-developer (Mentor):** POST /api/auth/admin/login nhận cả ADMIN và MENTOR, INTERN bị 403; cổng thực tập sinh giữ nguyên. Giao diện cổng Admin chấp nhận Mentor, các chức năng chỉ dành cho Admin (Quản trị hệ thống, tạo/xóa thực tập sinh) vẫn ẩn với Mentor. Viết MentorPortalTest (3 test) kiểm tra luồng Mentor giao task, chấm báo cáo, đánh giá cuối kỳ.
- **fullstack-developer (CI):** thêm JaCoCo vào pom.xml (chỉ báo cáo, không đặt ngưỡng) và workflow GitHub Actions .github/workflows/ci.yml: job backend dùng JDK 21 chạy mvn verify, ghi độ phủ vào trang tóm tắt và tải báo cáo JaCoCo lên làm artifact; job frontend chạy npm ci và npm run build.

Kết quả: 19/19 test pass; độ phủ dòng 79,33%, độ phủ nhánh 47,28% (đo cục bộ bằng JaCoCo).

7.5. Kiểm thử hồi quy, review lại và phát hành

Sau khi điều phối viên khởi động lại server với mã mới, ba agent chạy song song: tester chạy hồi quy trên server thật; code-reviewer review lại toàn bộ phần thay đổi; docs-

manager cập nhật năm tài liệu của ứng dụng (README, yêu cầu, hợp đồng API, kế hoạch kiểm thử, hướng dẫn client–server) theo hành vi mới.

Bảng 16. So sánh trước và sau (kiểm thử hồi quy)

Kiểm tra	Trước	Sau
Kiểm thử backend	3/3	26/26
Smoke API trên server thật	53/55	73/73
Đường dẫn không tồn tại	500	404
Sai HTTP method	500	405 (Allow: POST)
Mentor đăng nhập cổng Admin	403	200
Thực tập sinh đăng nhập cổng Admin	403	403
Lần đăng nhập sai thứ 6 (email thử)	401	429

code-reviewer kết luận "Sẵn sàng commit: Có". H2, H3, M1 được đánh giá đã sửa; H1 sửa phần giao diện, còn phần tài khoản mẫu giữ lại theo quyết định của sinh viên. Số phát hiện toàn dự án giảm từ 0/3/9/13 xuống 0/1/9/15 (Critical/High/Medium/Low): bỏ H2, H3, M1, M5; thêm 2 Medium và 2 Low mới.

Hai Medium mới nằm trong chính cơ chế khóa đăng nhập vừa viết ở vòng 1: N1, kiểm tra khóa và ghi nhận lần sai không nguyên tử nên khi gửi song song có thể thử vượt quá 5 lần; N2, bảng đếm lần sai không có giới hạn thật nên người chưa đăng nhập có thể làm server hết bộ nhớ. tester cũng độc lập phát hiện N2 khi đọc mã. Hai Low: N3, các lớp test dùng chung trạng thái khóa; N4, test của Mentor chưa chứng minh được việc lọc dữ liệu.

Sau khi commit thành 6 commit theo từng chủ đề và push, lần chạy CI đầu tiên trên GitHub Actions (run 41962378003) thành công cả hai job: backend chạy trên JDK 21 với 19/19 test pass, nghĩa là dự án đã được kiểm chứng trên Java 21 dù máy thực hiện chỉ có JDK 17; frontend build thành công.

7.6. Vòng 4–6: gia cố khóa đăng nhập, kiểm thử đột biến và sửa CORS

Vòng 4 (fullstack-developer): sửa N1–N4. Mỗi lần thử được giữ chỗ nguyên tử bằng ConcurrentHashMap.compute trước khi kiểm tra mật khẩu, nên dù gửi song song cũng chỉ có tối đa 5 lần kiểm tra mật khẩu trong 15 phút. Email dài hơn 254 ký tự bị từ chối với 400; bảng đếm có giới hạn 10.000 email, dọn mục hết hạn tối đa một lần mỗi phút. Một Clock được tiêm vào AuthService để test được mốc hết khóa 15 phút mà không phải chờ. Test HTTP chỉ dùng email ngẫu nhiên nên không còn khóa nhầm tài khoản mẫu; thêm test cô lập dữ liệu giữa hai Mentor. Kết quả: 25/25 test pass.

Kiểm chứng độc lập: code-reviewer xác nhận N1–N4 đã sửa nhưng tìm thêm hai Medium: N5, kẻ tấn công làm đầy bảng đếm bằng khoảng 300.000 request để đẩy mục đang khóa ra ngoài; N6, trên MySQL collation không phân biệt dấu nên các biến thể như

ádmin@gmail.com có thêm 5 lần thử mỗi biến thể. Song song, tester làm kiểm thử đột biến: cố ý đưa 6 lỗi vào bản sao mã nguồn (đảo thứ tự giữ chỗ, bỏ kiểm tra độ dài email, bỏ giới hạn bảng đếm, đổi cửa sổ 15 phút thành 30 phút, bỏ lọc dữ liệu Mentor, cho phép TODO → COMPLETED). Cả 6 lỗi đều bị test phát hiện (6/6).

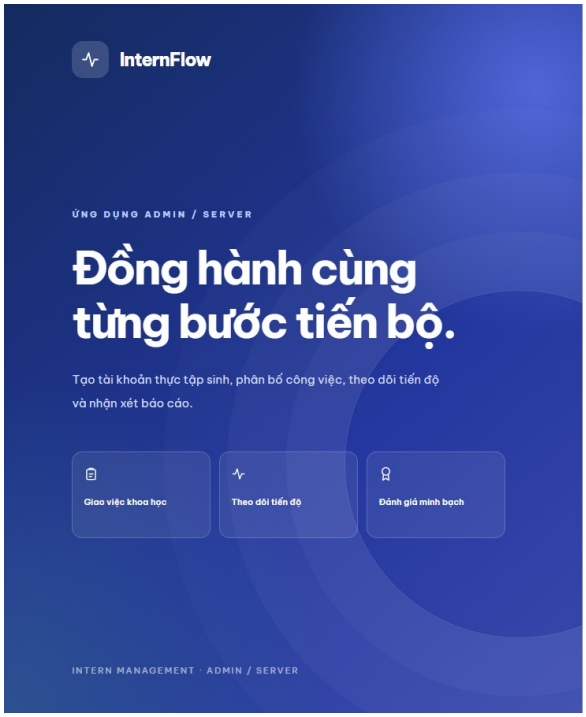
Vòng 5 (fullstack-developer): sinh viên quyết định chấp nhận N5 như rủi ro còn lại vì ứng dụng chạy trong LAN; comment trong mã và tên test được sửa cho đúng với khả năng thật. N6 được sửa: khóa đếm theo email lưu trong CSDL của tài khoản, nên mọi biến thể dùng chung một bộ đếm; test mới được chứng minh thất bại khi hoàn tác bản sửa. Kết quả: 26/26 test pass, số phát hiện toàn dự án còn 0/1/9/13, commit 34f73e4.

Vòng 6 (quay video, phát hiện lỗi thật): khi quay video, agent phát hiện cổng Admin chạy bằng Vite ở cổng 5174 bị 403 với mọi POST/PUT vì CORS mặc định chỉ cho phép cổng 5173. Lỗi này được giao cho một phiên Claude Code không giao diện (claude -p) điều phối ba subagent: fullstack-developer sửa application.properties và .env.example (DONE, 50 giây), tester chạy lại toàn bộ test 26/26 (DONE_WITH_CONCERNS: chưa có test riêng cho CORS, 48 giây), code-reviewer review diff (DONE_WITH_CONCERNS, 1 phút 31 giây). Toàn bộ phiên này được ghi lại và phát lại trong chương 2 của video. Độ phủ dòng đo lại là 80,32%.

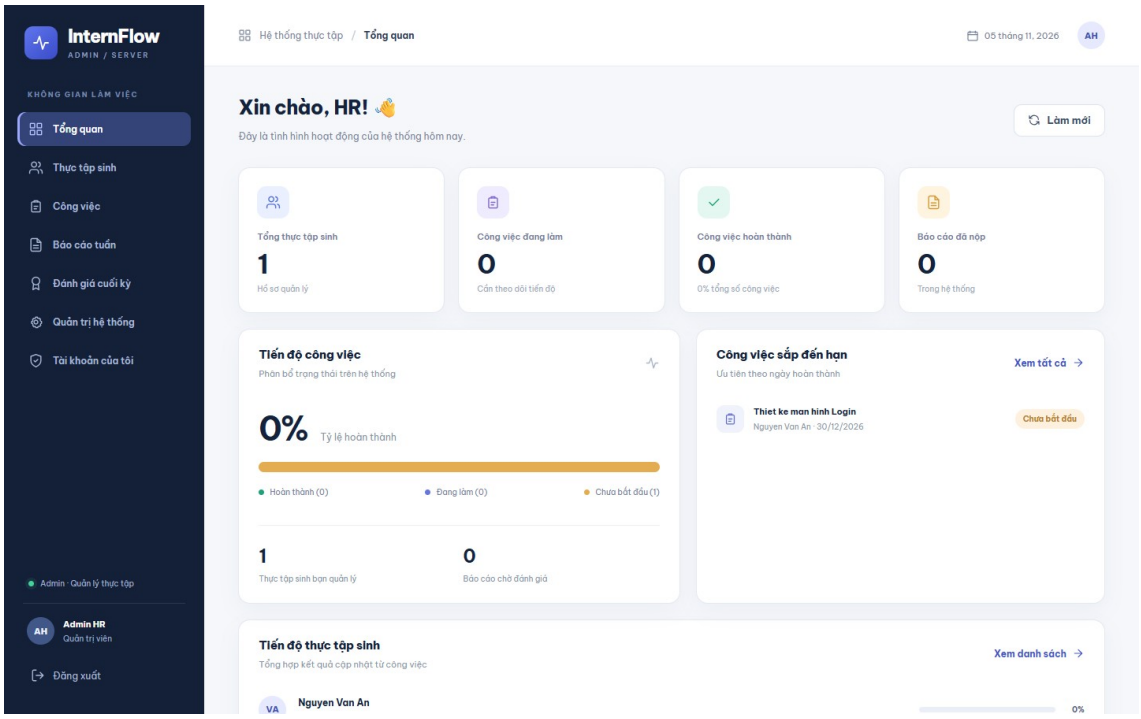
7.7. Ứng dụng chạy thật và video demo

Video demo docs/video/IMS_demo.mp4 dài 6 phút 36 giây (1440×900, H.264, 23,3 MB), chỉ có phụ đề tiếng Việt. Toàn bộ được quay và dựng tự động bằng ba script: src/record_demo_video.mjs điều khiển trình duyệt Edge chạy ẩn thao tác thật trên ứng dụng; src/record_agent_demo.mjs chạy thật các lệnh AgentKit và một phiên Claude Code điều phối ba subagent, rồi phát lại đầu ra trên màn hình kiểu terminal (có ghi rõ "phát lại phiên chạy thật, đã tua nhanh"); src/compose_demo_video.mjs ghép màn tiêu đề, các màn chương, hiệu ứng chuyển cảnh và màn tổng kết với số liệu đọc từ kết quả đo thật.

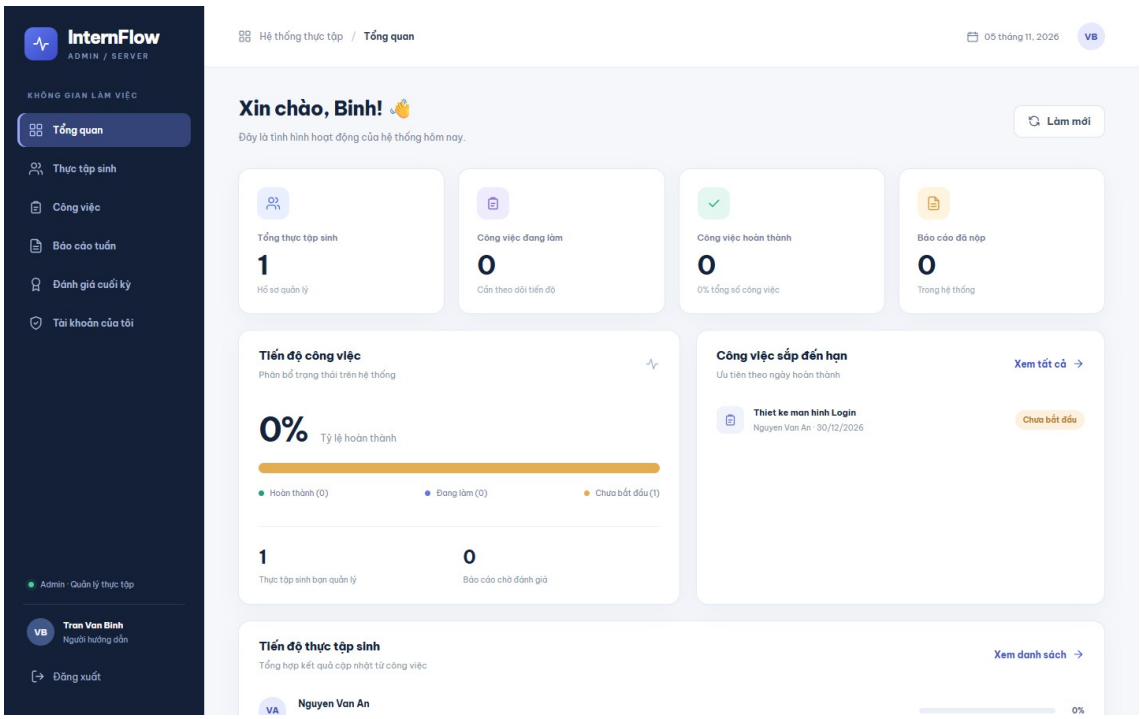
Các hình dưới đây được chụp tự động từ ứng dụng đang chạy (backend Spring Boot với MySQL 8.4, giao diện Vite) bằng script src/capture_report_figures.mjs, sau khi các thay đổi ở trên đã được áp dụng.



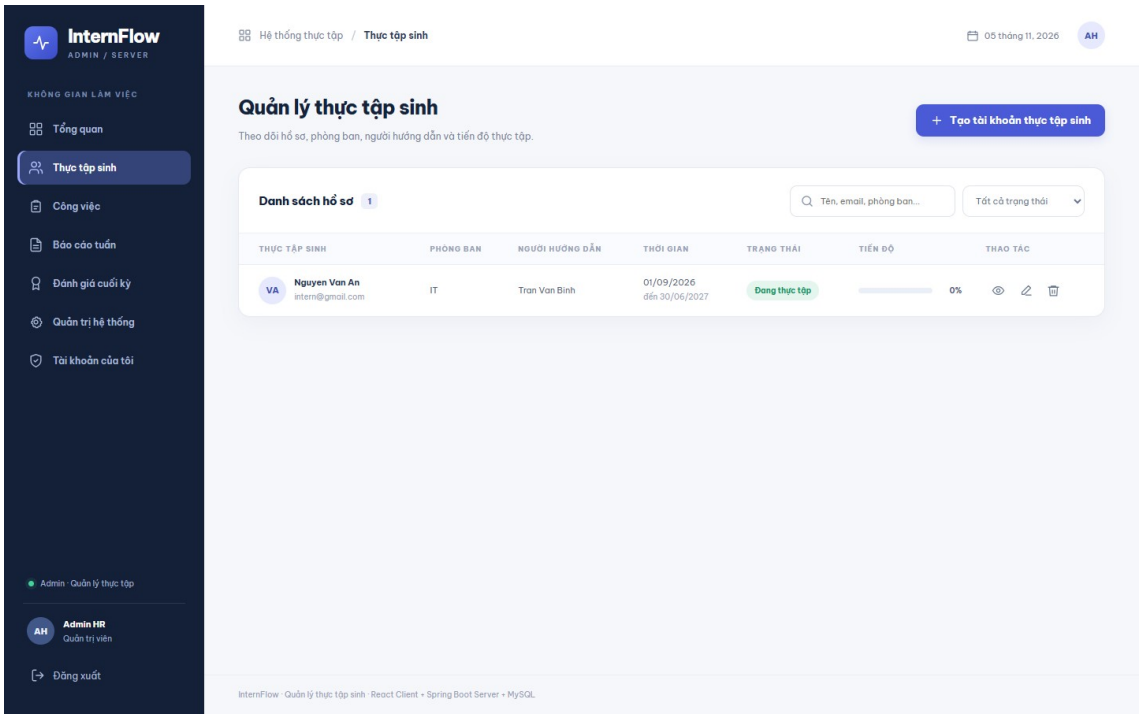
Hình 5. Màn hình đăng nhập cổng Admin/Mentor: không còn hiện mật khẩu mẫu



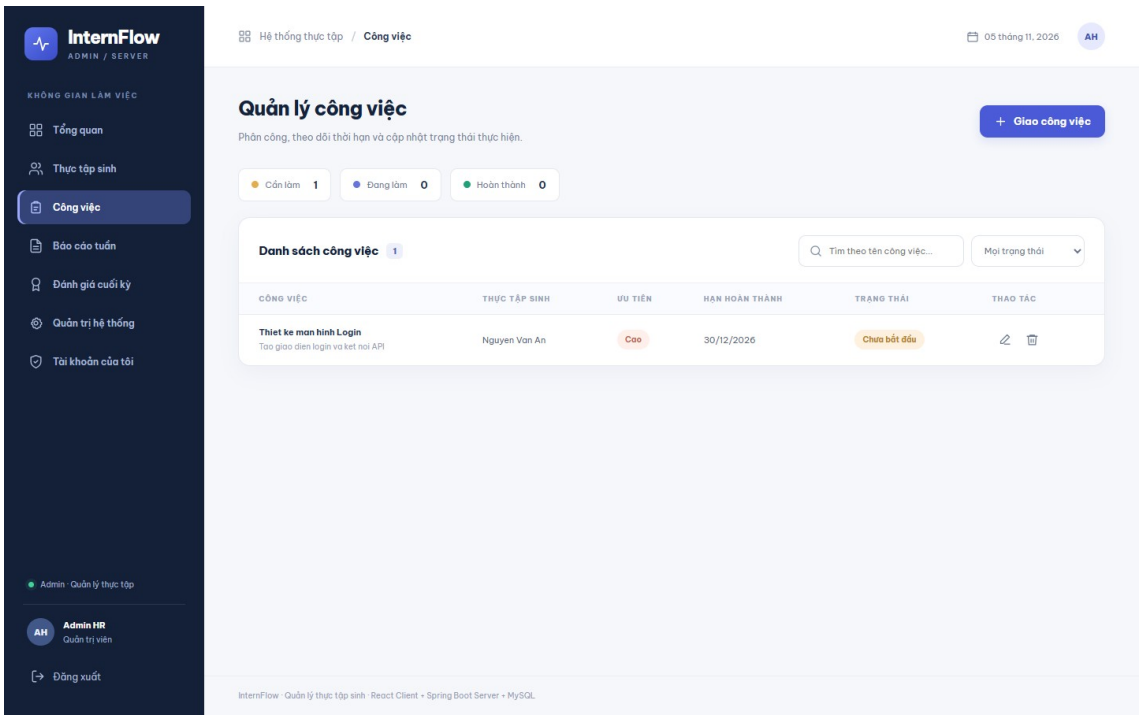
Hình 6. Dashboard của Admin



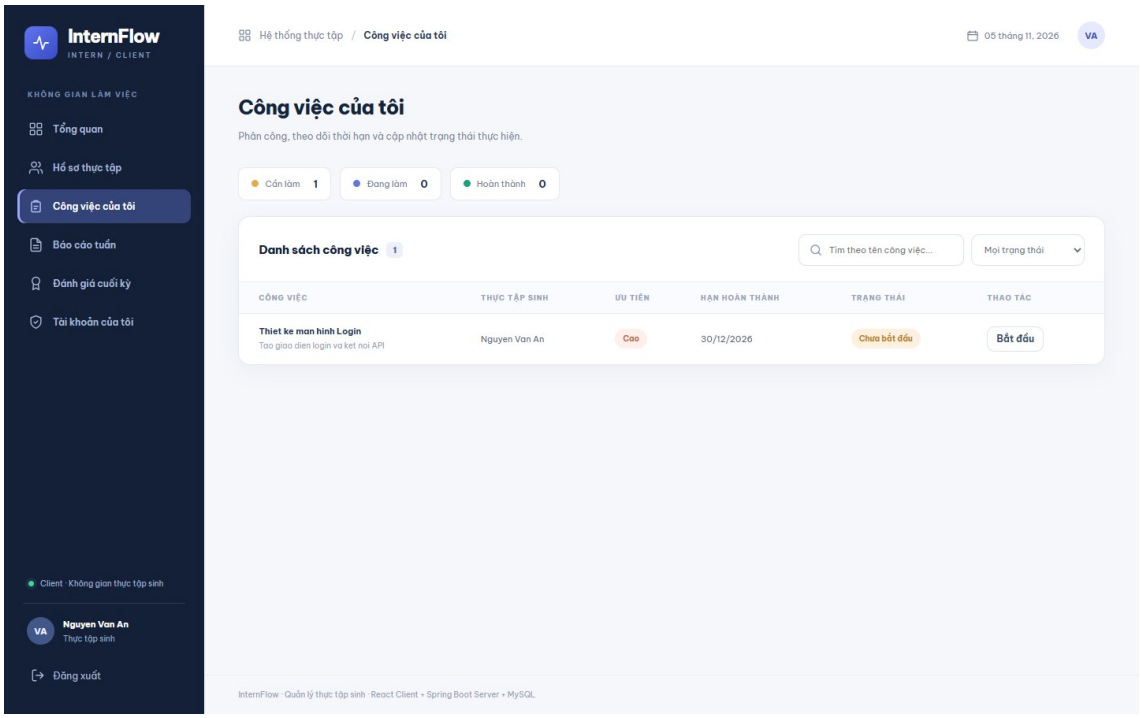
Hình 7. Mentor đăng nhập qua cổng Admin: mục Quản trị hệ thống bị ẩn



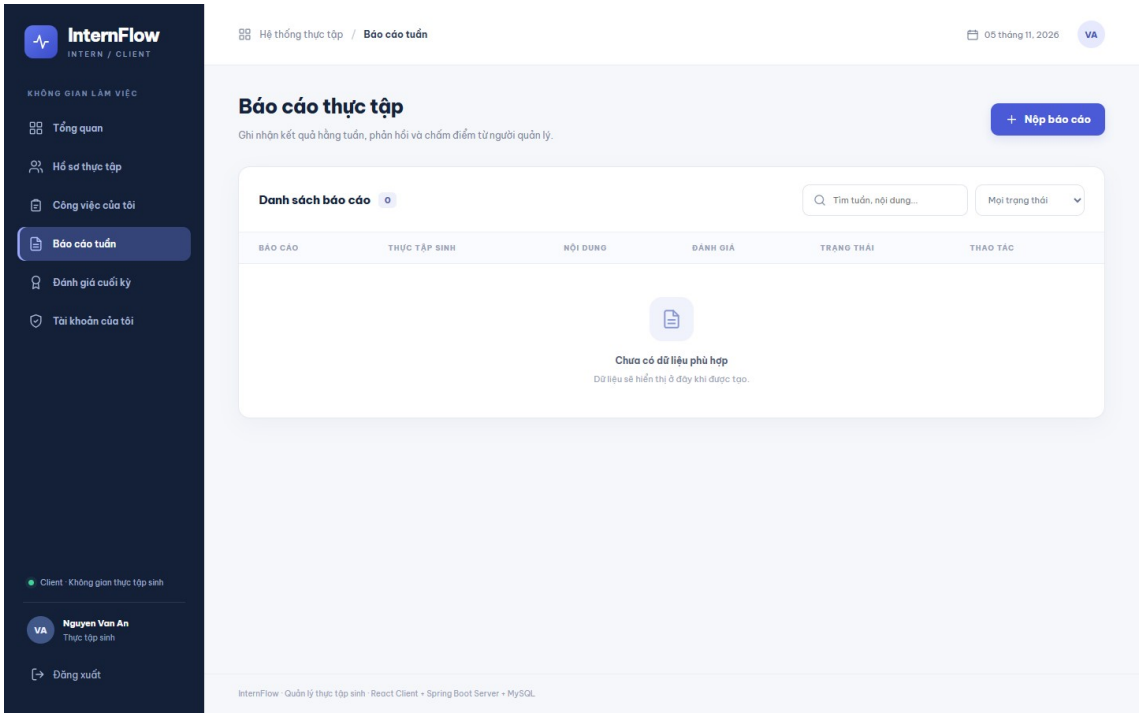
Hình 8. Danh sách thực tập sinh



Hình 9. Quản lý công việc



Hình 10. Cổng thực tập sinh: công việc của tôi



Hình 11. Cổng thực tập sinh: báo cáo tuần

8. Kết quả và đánh giá

8.1. Chỉ số trước và sau

Bảng 17. Chỉ số chất lượng trước và sau khi đội agent làm việc

Chỉ số	Đầu ngày	Cuối ngày
Số test tự động backend	3	26
TC04, TC05, TC08, TC10, TC13, TC15, TC18 có kiểm thử tự động	0/7	7/7
Smoke API trên server thật	53/55	73/73
Phát hiện review (Critical/High/Medium/Low)	0/3/9/13	0/1/9/13
Độ phủ dòng (JaCoCo)	chưa đo	80,32%
Kiểm thử đột biến (lỗi cố ý bị test phát hiện)	chưa đo	6/6
CI tự động	không có	GitHub Actions
Mentor dùng được giao diện	không	có

8.2. Nhận xét về cách làm việc với đội agent

- **Song song hiệu quả khi chia file rõ:** các vòng 1, 2, 3 đều có hai đến ba agent chạy cùng lúc mà không có xung đột ghi file nào, nhờ mục "file được phép sửa" trong phiếu giao việc.

- **Bằng chứng thay vì lời khẳng định:** mọi báo cáo agent đều kèm lệnh đã chạy và số liệu thô. Khi không chạy được (ví dụ hook chặn đọc thư mục build), agent ghi rõ là chưa kiểm chứng thay vì đoán.
- **Hai góc nhìn độc lập:** tester và code-reviewer cùng tìm ra lỗi H3 từ hai hướng; ở vòng 3, tester còn tìm ra một điểm yếu mới trong mã do chính developer viết ở vòng 1.
- **Con người giữ quyền quyết định:** xóa CSDL cũ, ghi đè cấu hình chung của máy, cách xử lý Mentor, có thêm CI hay không đều do sinh viên quyết định; agent chỉ đề xuất.
- **Rào chắn tự động có tác dụng thật:** hook đã chặn đọc file .env và chặn lệnh đụng thư mục build trong lúc làm việc; điều phối viên đổi cách làm theo đúng luật thay vì lách.

9. Video demo

Video docs/video/IMS_demo.mp4 dài 6 phút 36 giây, chỉ có phụ đề tiếng Việt, không lồng tiếng và không nhạc nền. Video được quay và dựng hoàn toàn tự động, không cắt ghép thủ công; mọi cảnh terminal là đầu ra của lệnh chạy thật được ghi lại kèm thời gian rồi phát lại (các đoạn chờ được tua nhanh và có nhãn ×N trên màn hình).

Bảng 18. Cấu trúc video demo

Thời điểm	Phần	Nội dung
00:00	Mở đầu	Màn tiêu đề: đề tài, môn học, sinh viên, công nghệ
00:06	Chương 1. Đội AI agent và AgentKit	Sơ đồ vai trò; ak --version và ak doctor; danh sách 16 agent trong .claude/agents; phần khai báo của tester.md; danh sách báo cáo trong plans/reports
00:58	Chương 2. Agent làm việc thật	Phát lại phiên claude -p sửa lỗi CORS cổng 5174: điều phối viên gọi lần lượt fullstack-developer, tester, code-reviewer, mỗi agent trả về dòng Status; git diff; mở báo cáo agent vừa viết; mvn verify 26/26; kết quả GitHub Actions bằng gh run view
04:06	Chương 3. Ứng dụng chạy thật	Admin đăng nhập và xem dashboard; Mentor đăng nhập qua cổng Admin, giao task; Thực tập sinh nhận task, chuyển trạng thái, nộp báo cáo tuần; Mentor chấm báo cáo
06:25	Tổng kết	26/26 test, CI xanh, đột biến 6/6, độ phủ dòng 80,32%

Lệnh dựng lại video nằm ở Phụ lục A.

10. Kết luận và hướng phát triển

10.1. Kết quả đạt được

- Tổ chức được một đội AI Agent đầy đủ vai trò PM, BA, Designer, Coder, Reviewer, Tester, DevOps và tài liệu bằng AgentKit, kèm quy trình, luật và rào chắn tự động.

- Chạy thật bảy vòng làm việc (0–6) trên mã nguồn IMS; sửa được các lỗi High H2 và H3, xử lý phần giao diện của H1, gia cố cơ chế khóa đăng nhập, sửa lỗi CORS; số test backend tăng từ 3 lên 26, độ phủ dòng 80,32%, kiểm thử đột biến phát hiện 6/6 lỗi cố ý; có CI trên GitHub Actions.
- Video demo 6 phút 36 giây được quay và dựng tự động, ghi lại cả đội agent làm việc thật lẫn ứng dụng chạy thật.
- Ứng dụng chạy end-to-end với MySQL, cả ba vai trò sử dụng được qua giao diện.

10.2. Hạn chế

- Mã nguồn IMS ban đầu không do agent viết từ đầu trong đợt này; agent tiếp quản một dự án có sẵn và cải tiến nó.
- Tài khoản mẫu với mật khẩu mặc định vẫn được tạo khi CSDL trống (sinh viên chấp nhận để phục vụ demo).
- Bộ đếm khóa đăng nhập nằm trong bộ nhớ, mất khi khởi động lại server và không chia sẻ giữa nhiều server; rủi ro N5 (làm đầy bảng đếm bằng khoảng 300.000 request) được chấp nhận và ghi rõ trong mã.
- Chưa có test riêng cho cấu hình CORS (tester và code-reviewer cùng nêu ở vòng 6).
- Bản desktop Electron chưa được đóng gói lại và chạy thử trong đợt này; đợt này chạy bản web.
- Máy thực hiện chỉ có JDK 17; JDK 21 được kiểm chứng trên CI.

10.3. Hướng phát triển

- Bắt buộc đổi mật khẩu ở lần đăng nhập đầu tiên và chỉ tạo tài khoản mẫu ở profile demo.
- Dùng HTTPS, chuyển lược đồ CSDL sang Flyway, tách App.jsx thành nhiều module và thêm lint, test cho frontend.
- Dùng /ak-vibe hoặc /ak-team để chạy trọn pipeline từ issue GitHub đến pull request đã qua CI một cách tự động.

Phụ lục A. Hướng dẫn chạy

```
# Backend với MySQL (mật khẩu đặt qua biến môi trường, không ghi vào kho mã)
cd app/backend
DB_USER=root DB_PASSWORD=<mat_khau_mysql> mvn -Djava.version=17 spring-boot:run

# Kiểm thử và báo cáo độ phủ
mvn -Djava.version=17 verify

# Giao diện web: cổng Thực tập sinh (:5173) và cổng Admin/Mentor (:5174)
cd app/frontend && npm install
npx vite --port 5173
VITE_PORTAL=admin npx vite --port 5174

# Sinh lại hình, video demo và báo cáo này (cần IMS_DEMO_PASSWORD, ffmpeg)
node src/capture_report_figures.mjs
node src/record_demo_video.mjs     # phần ứng dụng web
node src/record_agent_demo.mjs     # phần đội agent (chạy thật claude -p)
node src/compose_demo_video.mjs    # ghép thành docs/video/IMS_demo.mp4
python src/build_agent_report.py
```

Phụ lục B. Danh sách báo cáo do agent tạo ra

Bảng 19. Báo cáo agent

Agent	File báo cáo (thư mục plans/reports/)
code-reviewer	code-reviewer-261105-1002-ims-full-review.md
code-reviewer	code-reviewer-261105-1024-changes-review.md
code-reviewer	code-reviewer-261105-1040-lockout-hardening-review.md
code-reviewer	code-reviewer-261105-1121-cors-5174.md
docs-manager	docs-manager-261105-1024-app-docs-update.md
fullstack-developer	fullstack-developer-261105-1015-ci-jacoco.md
fullstack-developer	fullstack-developer-261105-1015-fix-error-handler-login-limit.md
fullstack-developer	fullstack-developer-261105-1015-mentor-admin-portal.md
fullstack-developer	fullstack-developer-261105-1031-demo-video.md
fullstack-developer	fullstack-developer-261105-1031-login-lockout-hardening.md
fullstack-developer	fullstack-developer-261105-1046-lockout-canonical-key.md
fullstack-developer	fullstack-developer-261105-1117-agent-demo-video.md
fullstack-developer	fullstack-developer-261105-1121-cors-5174.md
tester	tester-261105-1002-ims-test-report.md
tester	tester-261105-1015-business-rule-tests.md
tester	tester-261105-1024-regression-after-fixes.md
tester	tester-261105-1040-mutation-check.md
tester	tester-261105-1121-cors-5174.md

Phụ lục C. Phiếu giao việc thực tế

Trích phiếu giao việc điều phối viên gửi cho agent tester ở vòng 1 (rút gọn, giữ nguyên các mục bắt buộc):

```
Task: Add automated backend tests for the 7 IMS test cases that currently have none:
      TC04, TC05, TC08, TC10, TC13, TC15, TC18.
Read first: app/docs/05-testing.md, 02-ba-requirements.md, 04-api-contract.md,
      existing tests, controllers.
File you may create: BusinessRuleTest.java (one new file). Do NOT modify src/main,
      existing tests or pom.xml. Another agent is editing ApiExceptionHandler/AuthService
      in parallel; it adds a login lockout, so avoid repeated failed logins.
Rule: if actual behaviour contradicts the documented expectation, do NOT weaken the
      assertion; mark @Disabled("BUG: ...") and report it as a defect.
Acceptance: all 7 TCs covered and passing; Vietnamese report in plans/reports/.
End with: Status: DONE | DONE_WITH_CONCERNS | BLOCKED | NEEDS_CONTEXT
```